

Tối ưu thuật toán qua một lớp bài toán có tính đơn điệu

Tang Ze

Trường Trung học số 1 Trường Sa, Hồ Nam

2006

Nguồn gốc: Tối ưu thuật toán qua một lớp bài toán có tính đơn điệu

IOI China candidate team papers / 2006 / Tang Ze.doc

Tóm tắt

Khai thác đầy đủ quan hệ giữa dữ liệu thường là chìa khóa để xây dựng thuật toán tốt. Bài viết bắt đầu từ tính đơn điệu, thảo luận cách dùng một hàng đợi đặc biệt, cho phép chèn ở cuối và xóa ở cả hai đầu, để tối ưu một lớp bài toán có quan hệ đơn điệu. Qua đó, bài viết nhấn mạnh vai trò của việc nhận ra quan hệ ẩn trong dữ liệu khi thiết kế thuật toán.

Từ khóa: quan hệ dữ liệu; hàng đợi; tính đơn điệu.

1 Dẫn nhập

Trong nhiều bài toán, nếu ta khai thác được quan hệ dữ liệu ẩn bên trong đề bài và biến đổi một cấu trúc dữ liệu đơn giản cho phù hợp với quan hệ ấy, ta có thể tối ưu thuật toán mà không làm chương trình phức tạp đáng kể.

Hàng đợi là cấu trúc quen thuộc: một bảng tuyến tính vào trước ra trước, chỉ cho phép chèn ở một đầu và xóa ở đầu kia. Bài viết dùng một biến thể: hàng đợi cho phép chèn ở cuối, nhưng được xóa ở cả đầu và cuối. Nếu bài toán có quan hệ đơn điệu phù hợp, hàng đợi này giúp loại bỏ các trạng thái hoặc quyết định không còn khả năng trở thành tối ưu.

2 Ứng dụng trong quy hoạch động

Những ví dụ quen thuộc nhất của tối ưu bằng tính đơn điệu xuất hiện trong quy hoạch động. Một số bài có tính đơn điệu của quyết định, cho phép dùng hàng đợi đặc biệt để giảm một chiều tính toán.

2.1 Bài toán 1: Saw Mill

Bài *Saw Mill* từ CEOI 2004 có n cây cổ thụ trồng dọc một đường từ đỉnh núi xuống chân núi. Gỗ chỉ được vận chuyển xuống dốc. Ở chân núi đã có một xưởng cưa; cần chọn thêm hai vị trí đặt xưởng cưa trên đường sao cho tổng chi phí vận chuyển nhỏ nhất. Chi phí vận chuyển tỷ lệ với trọng lượng gỗ và khoảng cách.

Trước hết, đặt xưởng giữa hai cây kề nhau là vô nghĩa: ta có thể dời xưởng lên vị trí cây gần nhất phía trên, khi đó chi phí vận chuyển của các cây phía trên giảm còn chi phí của các cây phía dưới không tăng. Vì vậy chỉ cần xét vị trí tại các cây.

Đặt thêm một cây giả ở chân núi. Gọi W_i là tổng trọng lượng từ cây 1 tới cây i , D_i là khoảng cách từ cây 1 tới cây i . Với các tổng tiền xử lý này, chi phí vận chuyển một đoạn liên tiếp tới một xưởng ở cuối đoạn có thể tính trong $\mathcal{O}(1)$.

Giả sử xưởng thứ hai đặt ở cây i , còn xưởng thứ nhất đặt ở cây $j < i$. Dạng quy hoạch động cơ bản là

$$f_i = \min_{j < i} \{A_j + B(j, i)\},$$

trong đó A_j là chi phí phần phía trên đã xử lý và $B(j, i)$ là chi phí đưa đoạn còn lại về xưởng ở i . Tính trực tiếp mọi j cho mọi i có độ phức tạp $\mathcal{O}(n^2)$, không đủ nhanh với n lớn.

Điểm then chốt là quyết định tối ưu có tính đơn điệu: nếu tại vị trí i , quyết định tốt nhất là j , thì tại vị trí $i + 1$, quyết định tốt nhất không nhỏ hơn j . Bản gốc chứng minh điều này bằng cách so sánh hai quyết định $j < k$, chuyển các hạng chứa biến chung về một vế, rồi nhận ra hiệu giữa hai quyết định biến đổi đơn điệu theo i .

Khi đã có tính đơn điệu, ta duy trì một hàng đợi các quyết định còn có khả năng tối ưu. Hàng đợi q thỏa:

- các chỉ số quyết định tăng dần;
- mỗi quyết định trong hàng đợi tốt hơn quyết định đứng trước nó trên một đoạn liên tiếp của các trạng thái tương lai.

Trước khi tính f_i , nếu quyết định ở đầu hàng đợi đã không còn tốt hơn quyết định kế tiếp tại i , ta xóa nó. Khi đó đầu hàng đợi chính là quyết định tối ưu cho i . Sau khi tính xong, ta chèn quyết định mới i vào cuối; trong khi quyết định mới làm một số quyết định cuối hàng đợi trở nên vô ích trước khi chúng kịp tối ưu, xóa các quyết định ấy. Mỗi quyết định vào và ra hàng đợi nhiều nhất một lần, nên tổng thời gian duy trì hàng đợi là $\mathcal{O}(n)$.

Ví dụ này trực tiếp dùng tính đơn điệu của quyết định để giảm từ công thức quy hoạch động bậc hai xuống tuyến tính sau tiền xử lý.

2.2 Bài toán 2: Currency Exchange

Bài *Currency Exchange* từ BOI 2003 mô tả việc nhận tiền tệ A mỗi ngày, có thể dương hoặc âm, và được phép chọn ngày đổi toàn bộ A đang có sang B. Tỷ giá ngày i là 1 A đổi được i B, nhưng mỗi lần đổi phải trả thêm phí T B. Đến ngày N phải đổi hết A; cần tối đa hóa số B cuối cùng.

Nếu gọi S_i là tổng lượng A nhận từ ngày 1 tới i , công thức quy hoạch động tự nhiên chọn các ngày phân đoạn:

$$f_i = \max_{j < i} \{f_j + \text{gain}(j + 1, i) - T\}.$$

Tính trực tiếp vẫn là $\mathcal{O}(n^2)$. Tuy nhiên ở đây các A_i có thể âm, nên quyết định không đơn điệu một cách trực tiếp như bài Saw Mill.

Điều kiện quan trọng bị ẩn là tỷ giá tăng theo thời gian: A càng giữ lâu càng có giá trị hơn khi đổi sang B. Nếu trong một đoạn liên tiếp, tổng tiền A đang cầm không âm, thì không vội đổi thường không tệ hơn đổi sớm, vì vừa tránh mất thêm phí vừa hưởng tỷ giá cao hơn. Từ quan sát này, bản gốc nêu và chứng minh một mệnh đề: với một đoạn mà các tổng tiền tích lũy trung gian đều không âm cho tới khi gặp tổng âm đầu tiên, luôn tồn tại phương án tối ưu không đặt điểm đổi bên trong đoạn ấy.

Do đó ta có thể nén dãy A_i thành các đoạn. Bắt đầu từ ngày đầu tiên, gom các phần tử cho tới khi tổng đoạn trở thành âm; đoạn ấy có thể được xem như một phần tử mới. Lặp lại cho tới hết dãy. Dãy sau nén có đặc điểm quan trọng: trừ phần tử cuối có thể không âm, các phần tử còn lại đều âm. Khi đó các tổng tiền tổ tương ứng có tính đơn điệu cần thiết, và bài toán có thể dùng cùng kỹ thuật hàng đợi như Saw Mill. Phần tử cuối nếu đặc biệt thì xử lý riêng.

Điểm đáng chú ý là bài toán ban đầu không bộc lộ tính đơn điệu ở công thức trực tiếp. Chỉ sau khi khai thác điều kiện tỷ giá tăng và biến đổi dữ liệu, quan hệ đơn điệu mới xuất hiện.

3 Ứng dụng ngoài quy hoạch động

Tính đơn điệu và hàng đợi đặc biệt không chỉ dùng trong quy hoạch động. Nhiều bài toán duy trì cực trị trên một cửa sổ trượt cũng có thể tối ưu theo cách tương tự.

3.1 Bài toán 3: Travel

Bài *Travel* từ POI 2004 cho một đường tròn gồm n trạm xăng. Trạm i có p_i lít xăng, và từ trạm i tới trạm kế tiếp cần d_i lít. Xe bắt đầu với bình rỗng, khi tới trạm thì lấy toàn bộ xăng ở đó. Cần xác định với từng trạm, nếu xuất phát từ trạm ấy và đi theo một chiều cố định, có thể đi hết một vòng mà không bao giờ hết xăng hay không. Hai chiều kim đồng hồ và ngược chiều có bản chất giống nhau, nên chỉ cần phân tích một chiều.

Đặt

$$a_i = p_i - d_i.$$

Mở vòng tròn thành một dãy độ dài $2n$ bằng cách lặp lại $a_1, \dots, a_n$. Gọi

$$s_i = \sum_{k=1}^i a_k, \quad s_0 = 0.$$

Xuất phát từ trạm i đi được một vòng khi và chỉ khi mọi tổng nhiên liệu tích lũy trên đoạn $i..i+n-1$ đều không âm, tức

$$\min_{i \leq t \leq i+n-1} (s_t - s_{i-1}) \geq 0.$$

Điều này tương đương

$$\min_{i \leq t \leq i+n-1} s_t \geq s_{i-1}.$$

Cách đơn giản là mô phỏng từng điểm xuất phát, mất $\mathcal{O}(n^2)$. Dùng heap để duy trì giá trị nhỏ nhất trong cửa sổ trượt dài n cho ta $\mathcal{O}(n \log n)$. Dùng RMQ có thể đạt $\mathcal{O}(n)$ sau tiền xử lý, nhưng cài đặt phức tạp hơn.

Quan sát quan trọng là các cửa sổ cần xét có hai đầu mút cùng tăng dần:

$$[1, n], [2, n+1], \dots, [n, 2n-1].$$

Đây là đúng cấu trúc của bài toán minimum trên cửa sổ trượt. Ta duy trì một hàng đợi các chỉ số t có khả năng trở thành vị trí đạt giá trị nhỏ nhất của một cửa sổ trượt lai. Hàng đợi luôn có s_t tăng dần:

1. Trước khi xét cửa sổ bắt đầu ở i , xóa khỏi đầu hàng đợi các chỉ số đã nằm ngoài cửa sổ.
2. Khi thêm chỉ số mới ở cuối cửa sổ, xóa khỏi cuối hàng đợi mọi chỉ số có s không nhỏ hơn giá trị mới, vì chúng sẽ rời cửa sổ sớm hơn hoặc không tốt hơn chỉ số mới.
3. Đầu hàng đợi là vị trí có s_t nhỏ nhất trong cửa sổ hiện tại. So sánh giá trị đó với s_{i-1} để quyết định trạm i có khả thi hay không.

Mỗi chỉ số được chen một lần và xóa nhiều nhất một lần ở mỗi đầu theo đúng vòng đời của nó, nên thuật toán chạy trong $\mathcal{O}(n)$ và dùng $\mathcal{O}(n)$ bộ nhớ.

Bản gốc so sánh bốn cách giải:

Thuật toán	Bộ nhớ	Thời gian	Cài đặt
mô phỏng trực tiếp	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	rất dễ
heap	$\mathcal{O}(n)$	$\mathcal{O}(n \log n)$	đơn giản
RMQ	$\mathcal{O}(n)$	$\mathcal{O}(n)$	khó
hàng đợi đơn điệu	$\mathcal{O}(n)$	$\mathcal{O}(n)$	đơn giản

4 Kết luận

Ba ví dụ trên đều khai thác quan hệ đơn điệu ẩn trong dữ liệu và dùng một hàng đợi đặc biệt để tối ưu thuật toán. Do cấu trúc dữ liệu sử dụng rất đơn giản, độ khó cài đặt sau tối ưu hầu như không tăng so với cách làm ban đầu, nhưng độ phức tạp thời gian giảm đáng kể.

Điều này cho thấy một thuật toán tốt không nhất thiết phải dựa trên cấu trúc dữ liệu cao cấp. Nhiều khi, chỉ cần đào sâu điều kiện ẩn của bài toán, một cấu trúc đơn giản đã đủ để thực hiện tối ưu. Trong thi lập trình, các bài yêu cầu phát hiện quan hệ dữ liệu ẩn và vận dụng linh hoạt cấu trúc dữ liệu còn rất nhiều; càng biết quan sát và biến đổi, ta càng dễ xây dựng được thuật toán gọn và mạnh hơn.

Lời cảm ơn

Tác giả cảm ơn thầy Cao Liguó vì sự hướng dẫn trong quá trình viết bài, và cảm ơn bạn Zhou Gelin vì những hỗ trợ đã nhận được.

References

- [1] Liu Rujia, Huang Liang. *Nghệ thuật thuật toán và thi tin học*. Nhà xuất bản Đại học Thanh Hoa.
- [2] Luận văn đội tuyển quốc gia tin học năm 2005 của Zhou Yuan.